

Blockchain – Weekly Assignment 2

Github link: <https://github.com/sakhilebut5-star/Blockchain-Assignment-2>

Introduction

This assignment implements a simplified blockchain in python as a demonstration of how blocks are constructed, hashed, linked, and validated. The implementation stores the blocks in a blockchain and uses SHA-256 hashing together with proof-of-work mining process.

The structure of the blocks

The blocks are represented as dictionaries, each containing the six required fields, namely:

- Index: int
- Timestamp: str
- Transactions: list
- Previous hash: str
- Nonce: int
- Hash: str

Hash computation and canonical serialization

The hash calculation uses a canonical serialization. Before hashing, the block's own hash field is removed because including it would create a circular dependency: the hash would need to contain itself.

The hash must commit to all material fields so that changing any one of them is detectable.

Genesis block and subsequent block

Creating the blockchain. `__init__()` automatically mines the genesis block. The genesis block has index 0, no transactions, and a previous hash consisting of 64 zeroes. With a difficulty of 4, the resulting genesis block had nonce 388930 and hash:

`0000e3cddda4b35ed01d72914e6148ca1ad9ddd38562faeb914b8691aecc68cc`.

The four leading zeroes show that the proof-of-work requirement was satisfied. The two transactions were then added to the list:

Sakhile → Thabo: R250

Thabo → Lindiwe: R75.50

These transactions were included in block 1. The resulting block had index 1, nonce 79109, and its `previous_hash` was exactly equal to the genesis block's hash. This demonstrates that block 1 is cryptographically linked to the genesis block.

Tamper evidence demonstration

To demonstrate tamper evidence, the first transaction in Block 1 was changed after the block had already been mined. The amount was changed from R250 to R250 000 without changing the stored hash.

The original hash stayed as:

```
0000fb833fd5447ccc187be63e648e2af23ea95bef83981655c062e84a432005
```

But when the program recomputes the hash from the modified block, the result is:

```
f67db35c733e833a15167f837150315572d1733f3299ec2b57bd7f8647e88cc2
```

The two hashes do not match and consequently the chain-validation procedure reported that Block 1's stored hash did not match its recomputed hash, indicating that its contents had been altered after it was mined.

Furthermore, the implementation also shows why simply re-mining the altered block does not completely conceal the modification. After the transaction was changed, a new valid hash for block 1 was generated:

```
0000570a8451f85bf2ec55e92f8c95bc748e8c21bb44c90e797c6e8da3460bea
```

Because block 2 still contained the old hash for block 1 its `previous_hash`, the new block 1 hash and block 2's stored `previous_hash` did not match. The validation procedure consequently identified a broken chain link.

Conclusion

The implementation satisfies the main objectives of the Block Construction Lab. Blocks contain the required structural fields, and their material contents are deterministically serialized before being hashed with SHA-256. The genesis block and a subsequent mined block were successfully created, with the second block correctly referencing the genesis block's hash.

The tamper-evidence experiment demonstrates the central role of hashing in maintaining blockchain integrity. Altering a transaction changes the recomputed hash, immediately causing the stored hash to become invalid. Furthermore, re-mining the altered block does not repair the chain because subsequent blocks retain the original hash of their predecessor. The implementation therefore demonstrates both block-level tamper detection and chain-level linkage integrity.

